R: limpieza, manipulación y exploración de datos

1er encuentro: Instalación y primeros pasos

Jonatán Perren

26 de marzo de 2026

Instalación y configuración del entorno (R y RStudio)

Para trabajar con R, necesitamos instalar tres programas:

1. R: Es el lenguaje de programación y el motor de cálculo. Se descarga desde CRAN (The Comprehensive R Archive Network).
   `http://mirror.fcaglp.unlp.edu.ar/CRAN/`
2. RStudio: Es un Entorno de Desarrollo Integrado (IDE). Es una interfaz amigable que nos facilita escribir código, ver gráficos y gestionar archivos.
   `https://posit.co/download/rstudio-desktop`
3. Rtools (Solo Windows): Es un conjunto de herramientas necesarias para compilar paquetes desde su código fuente. Aunque al principio no lo notes, es fundamental tenerlo instalado para evitar errores futuros.
   `https://cran.r-project.org/bin/windows/Rtools/`

Instalación y configuración del entorno (R y RStudio)

El orden es clave. Primero instala R, luego Rtools (si usas Windows) y finalmente RStudio.

Podés encontrar más información sobre cómo en la pestaña **Software para el curso** del aula.

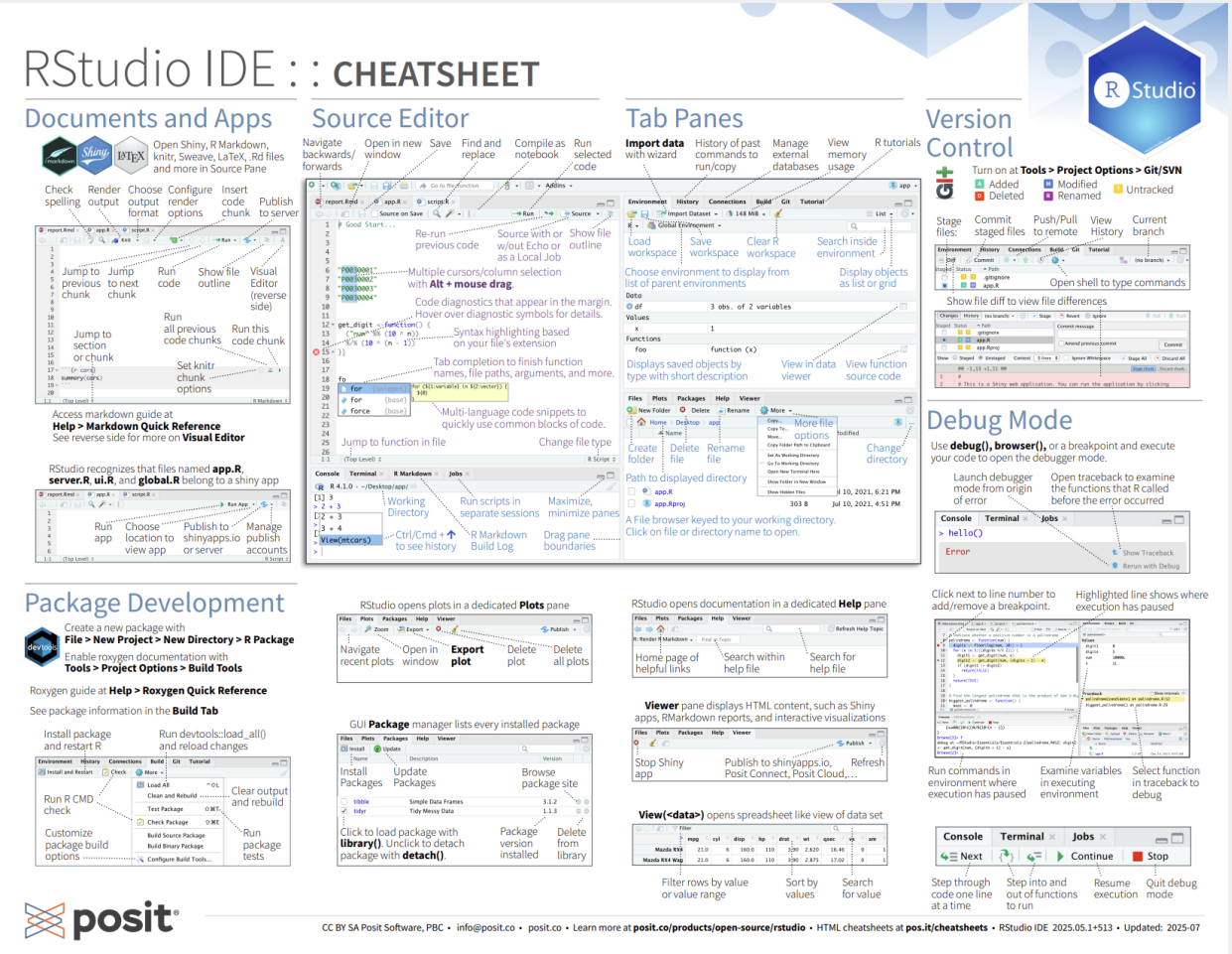
Introducción al entorno RStudio

Al abrir RStudio, verás la pantalla dividida en paneles:

1. Consola (Console): Es donde R ejecuta las instrucciones y muestra los resultados inmediatos. Si escribes `2 + 2` y das `Enter`, verás `4`.
2. Editor de Código (Source/Script): Aquí escribimos nuestros programas (scripts). Es como un bloc de notas inteligente. El código aquí no se ejecuta hasta que tú lo ordenas (usualmente con `Ctrl + Enter` para ejecutar un chunk).
3. Entorno (Environment): Muestra todos los objetos (variables, datos) que has creado y están disponibles en la memoria.
4. Archivos/Gráficos/Ayuda (Files/Plots/Help): Un panel multifunción para navegar carpetas, ver visualizaciones y consultar documentación.

Introducción al entorno RStudio

Puedes acceder a un cheatsheet de RStudio desde aquí:
`https://rstudio.github.io/cheatsheets/html/rstudio-ide.html`



Una versión en español está en la pestaña **Material de consulta rápida** en el aula virtual.

Buenas prácticas: Organización del Script

Un script es un fichero de texto en el que organizaremos el código. En ocasiones, un proyecto puede tener varios scripts involucrados.

Para crear un nuevo script iremos a `File > New file > R Script`.

Un script ordenado es un script feliz (y reproducible). Seguí estas reglas de oro:

1. Encabezado: Comienza siempre cargando todos los paquetes necesarios (`library( ... )`) en las primeras líneas.
2. Comentarios: Usa # para explicar el por qué de tu código, no el qué (el código ya dice qué hace).

```r
# install.packages("dplyr")
library(dplyr) # Para poder usar lag.

datos <- c(150, 100, 50, 15, 150)

#   Variación relativa de un vector <- Esto dice "qué". Evitar a menos
# que sea una definición extensa y requiera explicación detallada en
# cada paso.
variacion_relativa = function(x) {
  return ((x - lag(x)) / lag(x))
}

variacion_relativa(datos)
```

```
[1]         NA -0.3333333 -0.5000000 -0.7000000  9.0000000
```

Buenas prácticas: Organización del Script

También se puede implementar el típico docstring para documentar extensamente una aplicación. Pero, como estamos aprendiendo, no es necesario tanto rigor. Esto es fundamentalmente útil cuando se trabaja con proyectos muy extensos, colaborativos o paquetes.

```r
#' Calcular el doble de un número n
#' Toma un argumento numérico y devuelve el doble.
#' @param n El número a doblar
#' @return El doble de n
#' @examples
#' doble_de(15)
doble_de <- function(n) return(2 * n)
```

```
> doble_de(10)
[1] 20
```

Buenas prácticas en general

Asegúrate siempre de guardar tu trabajo en un fichero y de fijar tu directorio de trabajo donde este esté. Para conocer tu directorio de trabajo ejecuta

```
getwd() #get working directory
```

Para configurar el directorio de trabajo se puede ejecutar

```
setwd("C: ... ") #set working directory
```

con tu dirección deseada, o en RStudio desde la pestaña
`Session > Set Working Directory > To Source File Location`

Cuando se selecciona esa opción se puede apreciar en la consola que se ha ejecutado el comando antes mencionado en la dirección del fichero de trabajo.

¿Por qué ajustar el directorio de trabajo?

El motivo por el cual se ajusta el directorio de trabajo es que R sepa cuál es el directorio raíz desde donde se cargan o dónde debe exportar los archivos.

Objetos básicos y asignación de variables

Podemos usar R como una calculadora. La consola de R actúa como un entorno interactivo donde podemos evaluar expresiones matemáticas inmediatamente. Es el uso más básico del lenguaje.

Los operadores aritméticos estándar funcionan como se espera:

- Suma: `+`
- Resta: `-`
- Multiplicación: `*`
- División: `/`
- Potenciación: `^`

```r
5 + 5
```

```
[1] 10
```

```r
5 * 5
```

```
[1] 25
```

```r
5 / 5
```

```
[1] 1
```

```r
5 ^ 5
```

```
[1] 3125
```

También disponemos de funciones matemáticas predefinidas. Una función se llama por su nombre seguido de paréntesis `nombre_funcion(argumento)`.

```r
sqrt(9) #raíz cuadrada
```

```
[1] 3
```

```r
abs(-5) #valor absoluto
```

```
[1] 5
```

```r
log(10) #logaritmo natural
```

```
[1] 2.302585
```

```r
factorial(4) #factorial
```

```
[1] 24
```

Consultar definiciones

Para consultar la definición y el comportamiento de una función, el contenido o variables de un marco de datos, podemos usar el prefijo ? antes del elemento.

Se puede convertir en tu mejor compañero en R

También se pueden hacer comparaciones. Se pueden usar operadores como >, <, >= (≥), <= (≤) entre otros. Quizá te sorprenda que las comparaciones de igualdad se hacen con == (y no con =) y las de desigualdad con != (≠). El operador de conjunción es & y el de disyunción es |.

```
3 < 5
3 < 3
3 ≤ 3
```

```
[1] TRUE
[1] FALSE
[1] TRUE
```

```
edad ← 30; edad == 30
```

```
[1] TRUE
```

```
TRUE & FALSE
TRUE | FALSE
```

```
[1] FALSE
[1] TRUE
```

Podemos combinar operaciones respetando la precedencia matemática o usando paréntesis para agrupar.

```
(5 + 5) * 2
```

```
[1] 20
```

```
5 + 5 * 2
```

```
[1] 15
```

```
sqrt(sqrt(9) + 3 * 2)
```

```
[1] 3.0
```

```
n ← 10
(n ≠ 10) | ((n ≥ 10) & (n ≤ 20))
```

```
[1] TRUE
```

Parada técnica

En una diapositiva anterior teníamos

```r
# install.packages("dplyr")
library(dplyr)

datos <- c(150, 100, 50, 15, 150)

variacion_relativa = function(x) {
  return ((x - lag(x)) / lag(x))
}

variacion_relativa(datos)
```

```
[1]         NA -0.3333333 -0.5000000 -0.7000000  9.0000000
```

Intenta averiguar:

1. ¿Por qué es necesario definir funciones en ocasiones?
2. ¿Cómo funciona el mecanismo para la definición de nuevas funciones?
3. Licenciados y licenciadas en economía... Averigüen qué hace `lag()`
   ¿Viene incluido en R? ¿Cómo es que lo estoy usando?

Intenten esquematizar el comportamiento de esta función. ¿Qué dato recibe? ¿Cómo lo procesa?

Asignación en variables

En R, guardamos valores dentro de objetos (variables) para usarlos después. La ventaja de esto es que se puede reutilizar la cantidad de veces que uno quiera y, si necesita cambiarla, lo hace desde un solo lugar. Para esto usamos el operador de asignación <- (Mi typeface tiene las ligaduras activadas, por lo que en lugar de ver <- verás ←).

```
valor ← 7
valor + 8
```

```
[1] 15
```

```
valor ^ valor
```

```
[1] 823543
```

```
valor ← 5 #Si volvemos a cambia la primera linea
valor + 8
```

```
[1] 13
```

Aunque el signo igual = también funciona, por convención y buenas prácticas en R se prefiere ←.

```r
# Asignamos el valor 10 al objeto 'x'
x ← 10

# Ahora podemos usar 'x' en operaciones
x * 2
```

```
[1] 20
```

```r
x = 10 #Funciona, pero NO se recomienda
x * 2
```

```
[1] 20
```

```r
10 → x #Funciona, pero NO se recomienda
x * 2
```

```
[1] 20
```

Nombre de los objetos

R es muy flexible, pero seguir convenciones hace que tu código sea legible para tu «yo» del futuro (y para tu profe que te va a tener que corregir 🙃).

Reglas duras (condiciones necesarias)

- Deben comenzar con una letra.
- No pueden contener espacios (usa guion bajo _).
- R distingue mayúsculas de minúsculas (`Dato` es distinto de `dato`).

Convenciones de estilo (condiciones recomendadas)

- snake_case (recomendado por tidyverse): `mi_variable`, `promedio_edad`. Es fácil de leer.
- camelCase: `miVariable`, `promedioEdad`. Común en **otros** lenguajes.
- dot.case: `mi.variable`. Histórico en R, pero a veces confunde con métodos de objetos o con variables ocultas.

Estado	`Variable`	Razón
Recomendado	`ingreso_total`	Descriptivo y usa sc.
Correcto	`IngresoTotal`	Estilo UpperCamelCase.
Correcto	`ingresoTotal`	Estilo LowerCamelCase.
Correcto	`ingreso.total`	Estilo antiguo.
Incorrecto	`1ingreso`	Empieza con número.
Incorrecto	`ingreso total`	Contiene espacios.
Incorrecto	`ingreso-total`	El guion `-` es un operador.
Evitar	`c, t, df`	Nombres de funciones del sistema o muy ambiguos.
Evitar	`x, cosa`	No describe el contenido del objeto.
Evitar	`prom_edad_2023`	Muy largo; se prefiere un equilibrio como `promedio_edad`.

Vectores y tipos de datos (numéricos, caracteres, lógicos)

Comprender los tipos de datos es fundamental cuando trabajamos en R, especialmente porque más adelante estaremos procesando data frames (marco de datos) que contienen múltiples variables. Cada variable en un data frame tiene un tipo específico (números, texto, categorías, etc.), y conocer estos tipos nos permitirá aplicar las operaciones correctas y evitar errores al procesarlos.

Del mismo modo, es esencial entender los vectores, ya que la información en R está alojada en esta estructura: cada columna de un dataframe es un vector, y las operaciones que apliquemos sobre ellos determinarán el análisis que podamos realizar.

Acá podemos ver cómo es que teniendo el tipo de dato correcto (`Date`) podemos extraer información útil.

```r
# install.packages("lubridate")
library(lubridate) #Para manejar fechas

fechas <- c("2023-01-01", "2025-03-02",
            "2023-08-07", "2023-11-10")

class(fechas)
```

```
[1] "character"
```

```r
fechas <- as_date(fechas) #Cambiamos el tipo de datos a fecha.

class(fechas)
```

```
[1] "Date"
```

```r
year(fechas)  #Obtener años
month(fechas) #Obtener meses
day(fechas)   #Obtener días
```

```
[1] 2023 2025 2023 2023
[1]  1  3  8 11
[1]  1  2  7 10
```

El vector es la estructura de datos fundamental en R. Un vector es una colección ordenada de elementos **del mismo tipo** (Recodá esto para mas adelante). Para cargar un vector por extensión, se utiliza la instrucción `c(item1, item2, ...)`.

```r
texto   <- c("A", "b", "C", "5")
enteros <- c(1, 5, 6, 8)
mixto   <- c("1", "2", 3, 4) #!!!

#   Esto último NO devuelve error, pero tampoco almacena los datos
# como se los cargamos. ¿Qué es lo que está haciendo?
```

Tipos principales

En R existen varios tipos de datos, incluyendo factores (categóricos), fechas (Date/POSIXct), complejos (números complejos) y otros tipos especializados que veremos más adelante en el curso. Incluso, cuando aprendas a usar listas, podrías crear tus propios tipos de datos.

Por ahora, comenzaremos con los tres tipos más frecuentes en análisis de datos:

1. Lógicos
2. Numéricos
3. Cadena de texto

Tipos principales: Lógicos

Solo toman valores TRUE o T (Verdadero) o FALSE o F (Falso).

Ejemplos de variables estadísticas

- Aprobó/Desaprobó un examen.
- Tiene empleo/No tiene empleo.
- Posee vivienda propia (Sí/No).
- Está inscripto en un programa social.
- Respondió a una encuesta.

```r
es_mayor_de_edad = TRUE
es_mayor_de_edad
```

```
[1] TRUE
```

```r
aprobado <- c(TRUE, FALSE, TRUE, TRUE) #Vector de aprobados
aprobado
```

```
[1]  TRUE FALSE  TRUE  TRUE
```

Otras formas de generar valores lógicos

Además de asignar directamente TRUE o FALSE, los valores lógicos se generan frecuentemente como resultado de operaciones

Operadores de comparación

Comparan valores y devuelven TRUE o FALSE.

```r
# Comparaciones numéricas
edad ← 25
edad ≥ 18           # ¿Es mayor o igual a 18?
```

```
[1] TRUE
```

```r
edad == 30          # ¿Es igual a 30? (notá que es == y no =)
```

```
[1] FALSE
```

```r
# Comparaciones con vectores (elemento a elemento)
notas  ← c(8, 5, 7, 4, 9)
aprobo ← notas ≥ 6  # Vector lógico indicando quiénes aprobaron
aprobo
```

```
[1] TRUE FALSE TRUE FALSE TRUE
```

La lógica detrás de por qué funciona este último ejemplo la veremos luego.

Funciones is.*

Verifican el tipo de un objeto.

```r
x <- 42
is.numeric(x)        # ¿Es numérico?
```

```
[1] TRUE
```

```r
is.character(x)      # ¿Es texto?
```

```
[1] FALSE
```

```r
is.logical(x)      # ¿Es lógico?
```

```
[1] FALSE
```

```r
y <- "Hola"
is.character(y)
```

```
[1] TRUE
```

Otras funciones útiles

```r
# Verificar valores faltantes
datos <- c(10, NA, 30, NA, 50)

is.na(datos)
```

```
[1] FALSE TRUE FALSE TRUE FALSE
```

```r
# Verificar pertenencia ( %in% )
provincias <- c("Santa Fe", "Buenos Aires", "Córdoba")

"Santa Fe"  %in% provincias
"Mendoza"   %in% provincias
"Cordoba"   %in% provincias
```

```
[1] TRUE
[1] FALSE
[1] FALSE
```

```r
# Verificar múltiples condiciones
valores <- c(5, 15, 25, 35)

(valores > 10) & (valores < 30) # TRUE solo para 15 y 25
```

Tipos principales: Numéricos

Números enteros o racionales.

Ejemplos de variables estadísticas

- Edad (años)
- Ingreso mensual (pesos)
- Cantidad de hijos
- Temperatura (°C)
- Precio de un producto
- Peso (kg)
- Calificación de un examen (0-100)
- Años de educación
- Tasa de desempleo (%), PIB per cápita.

```
mi_edad ← 30
alturas ← c(1.75, 1.80, 1.65)
edades  ← c(25, 30, 22)
```

Tipos principales: Caracteres

Texto. Siempre van entre comillas.

Ejemplos de variables estadísticas

- Nombre de una persona,
- Provincia de residencia (aunque pueden considerarse factores),
- Código postal,
- Ocupación laboral,
- Marca de un producto (también factor),
- Dirección de email,
- Respuesta a pregunta abierta,
- DNI o CUIT (aunque sean números, se tratan como texto porque no tiene sentido sumarlos).

```r
nombre_del_curso <- "Curso de R"
nombres          <- c("Ana", "Juan", "Pedro")
actividad        <- "curso de R"
encargado        <- "Jonatán Perren"

paste("El encargado del", actividad, "es", encargado) #!
```

¿Para que sirve `paste()`? ¿Qué diferencia hay con `paste0()`?

Será importante más adelante poder identificar el tipo de dato que recibimos y el que tenemos para evitar algunos errores.

```
num1 ← 5
num2 ← "8"
num1 + num2 #Note que num2 no es de tipo numérico
```

```
Error en num1 + num2: argumento no-numérico para operador binario
```

Aunque algunos posibles problemas se solucionan por coerción. La coerción ocurre cuando un tipo de dato se convierte en otro para complacer a una función. La conversión ocurre en este orden:

$$\texttt{logical} \rightarrow \texttt{integer} \rightarrow \texttt{numeric} \rightarrow \texttt{character}$$

Por ejemplo, supongamos que tenemos `b1` ← `TRUE`, `b2` ← `TRUE` y `b3` ← `FALSE`. La operación suma (bajo el operador `+`) no está definida para datos lógicos. Aun así, observe el siguiente bloque

```
b1 ← TRUE; b2 ← TRUE; b3 ← FALSE

b1 + b2 + b3
```

```
[1] 2
```

Lo que ocurre es que la coerción convierte los datos en enteros (`integer`) y aplica la suma. Por defecto, `TRUE` se convierte en `1` y `FALSE` se convierte en `0`.

Sabiendo que la función `paste()` recibe cadenas y las concatena. Observe el siguiente bloque

```
entero ← 30
cadena ← "Mi edad es"
paste(cadena, entero)
```

```
[1] "Mi edad es 30"
```

En este caso, `entero` se convierte por coerción en `character` y se concatena.

También vimos a la coerción en acción en un ejemplo anterior cuando construimos un vector con datos de tipo distinto.

```r
mixto   <- c("1", "2", 3, 4) #!!!

mixto
```

```
[1] "1" "2" "3" "4"
```

Notá como se han convertido todos en caracteres. Realmente no tiene nada de **`mixto`**.

Es posible convertir generalmente datos de un tipo a otro con funciones como `as.logical()`, `as.integer()`, `as.numeric()` y `as.character()`, entre otros que veremos más adelante. Si volvemos al ejemplo que fallaba antes

```
num1 <- 5
num2 <- "8"
num2 <- as.integer(num2) #Ahora num2 es entero.
num1 + num2
```

```
[1] 13
```

Estas funciones serán de suma utilidad cuando tengamos que limpiar marcos de datos.

¿Cuál creés que serán los resultados en los siguientes dos vectores?

```
v1 <- c(1, 2, TRUE, 0)
v2 <- c(1, 2, "TRUE", 0)
v3 <- c(1, 2, TRUE, "0")
```

Formas de crear vectores

Por extensión

Enumerar elementos explícitamente. Se utiliza la función `c()`

```
mis_numeros <- c(1, 5, 10)
mis_numeros
```

```
[1] 1 5 10
```

Por secuencia

Rangos de enteros consecutivos. Se carga mediante `:` entre el entero inferior y el entero superior.

```
uno_al_diez        <- 1:10
dos_al_menos_siete <- 2:-7
uno_al_diez
dos_al_menos_siete
```

```
[1]  1  2  3  4  5  6  7  8  9 10
[1]  2  1  0 -1 -2 -3 -4 -5 -6 -7
```

Por progresión

Como el anterior, pero con un paso definido. Se carga mediante la función `seq()`.

```
lista ← seq(from = -3, to = 10, by = 2)
lista #De -3 hasta 10 de 2 en 2.
```

```
[1] -3 -1  1  3  5  7  9
```

```
lista ← seq(from = 0, to = 1, length.out = 21)
lista #21 elementos uniformemente distribuidos entre 0 y 1
```

```
[1] 0.00 0.05 0.10 0.15 0.20 0.25 0.30
[8] 0.35 0.40 0.45 0.50 0.55 0.60 0.65
[15] 0.70 0.75 0.80 0.85 0.90 0.95 1.00
```

Por repetición

Repite elementos un número de veces. Se carga mediante la función `rep()`

```
lista ← rep(5, times = 3)
lista #Repite 5 tres veces.
```

```
[1] 5 5 5
```

```
lista ← rep(0:2, length.out = 5)
lista
```

```
[1] 0 1 2 0 1
```

```
lista ← rep(0:2, each = 2)
lista
```

```
[1] 0 0 1 1 2 2
```

Por distribución de probabilidad

Por muestreo aleatorio. Varias funciones en R que tienen el prefijo `r*` devuelven un vector de datos. Tambien se pueden tomar muestras aleatorias de un vector mediante la función `sample()`.

```
lista ← 1:100
muestra ← sample(lista, size = 10)
muestra #Muestra de tamaño 10 sin repetición
```

```
[1] 81 80 44 57 28 96 30 84 65 50
```

```
booleanos ← c(TRUE, FALSE)
muestra ← sample(booleanos, size = 10, replace = TRUE)
muestra #Muestra de tamaño 10 con repetición
```

```
[1] FALSE FALSE  TRUE FALSE FALSE
[6] FALSE FALSE  TRUE FALSE  TRUE
```

Formas de crear vectores

Averigüá cuáles son las funciones que trae R por defecto para tomar muestras de distribuciones de probabilidad conocidas.

Operadores Lógicos y Relacionales

Podemos hacer preguntas a nuestros datos. El resultado siempre será un vector lógico (`TRUE` o `FALSE`).

Tipo	Operación	Operador	Ejemplo de Uso	Resultado
Unario	Negación (NOT)	`!`	`!TRUE`	`FALSE`
		`!`	`!(c(TRUE, FALSE))`	`c(FALSE, TRUE)`
	Val. faltantes	`is.na()`	`is.na(c(1, NA, 3))`	`c(F, T, F)`

Operadores y operaciones más frecuentes

Tipo	Operación	Op.	Ejemplo de Uso	Resultado
Relación	Igual a	`==`	`5 == 5`	`TRUE`
		`==`	`c(1, 2, 3) == 2`	`c(FALSE, TRUE, FALSE)`
	Distinto de	`≠`	`5 ≠ 3`	`TRUE`
		`≠`	`c(1, 2, 3) ≠ 2`	`c(TRUE, FALSE, TRUE)`
	Mayor que	`>`	`5 > 3`	`TRUE`
		`>`	`c(1, 5, 3) > 2`	`c(FALSE, TRUE, TRUE)`
	Menor que	`<`	`3 < 5`	`TRUE`
		`<`	`c(1, 5, 3) < 4`	`c(TRUE, FALSE, TRUE)`
	Mayor o igual	`≥`	`5 ≥ 5`	`TRUE`
		`≥`	`c(4, 5, 6) ≥ 5`	`c(FALSE, TRUE, TRUE)`
	Menor o igual	`≤`	`3 ≤ 5`	`TRUE`
		`≤`	`c(4, 5, 6) ≤ 5`	`c(TRUE, TRUE, FALSE)`

Operadores y operaciones más frecuentes

Tipo	Operación	Op.	Ejemplo de Uso	Resultado
Lógico	AND (Y)	`&`	`TRUE & FALSE`	`FALSE`
		`&`	`(c(T, T, F) & c(T, F, T))`	`c(T, F, F)`
	OR (O)	`|`	`TRUE | FALSE`	`TRUE`
		`|`	`(c(T, T, F) | c(F, F, T))`	`c(T, T, T)`
Otros	Pertenencia	`%in%`	`"a" %in% c("a", "b")`	`TRUE`
		`%in%`	`c(1, 3) %in% c(1, 2)`	`c(T, F)`

Operadores y operaciones más frecuentes

Operadores numéricos

R funciona como una calculadora científica vitaminada. Además de las operaciones básicas, tenemos funciones para casi todo.

Operación	Operador/Función	Ejemplo de Uso	Resultado
Suma	`+`	`c(4, 9) + 2`	`c(6, 11)`
Resta	`-`	`c(10, 5) - 3`	`c(7, 2)`
Multiplicación	`*`	`c(3, 5) * 2`	`c(6, 10)`
División	`/`	`c(10, 20) / 5`	`c(2, 4)`
Potencia	`^`	`c(2, 3) ^ 2`	`c(4, 9)`
Raíz cuadrada	`sqrt()`	`sqrt(c(4, 9, 16))`	`c(2, 3, 4)`
División entera	`%/%`	`10 %/% 3`	`3`
Resto (módulo)	`%%`	`10 %% 3`	`1`

Operadores y operaciones más frecuentes

Operación	Op./Función	Ejemplo de Uso	Resultado
Logaritmo nat.	`log()`	`log(c(1, 2.718))`	`c(0, 1)`
Exponencial	`exp()`	`exp(c(0, 1))`	`c(1, 2.718 ... )`
Valor absoluto	`abs()`	`abs(c(-5, 3))`	`c(5, 3)`
Redondeo	`round()`	`round(3.14159, 2)`	`3.14`
Redondeo arriba	`ceiling()`	`ceiling(3.14)`	`4`
Redondeo abajo	`floor()`	`floor(3.99)`	`3`
Máximo	`max()`	`max(c(1, 5, 3))`	`5`
Mínimo	`min()`	`min(c(1, 5, 3))`	`1`
Suma total	`sum()`	`sum(c(1, 2, 3))`	`6`
Promedio	`mean()`	`mean(c(2, 4, 6))`	`4`
Mediana	`median()`	`median(c(1, 2, 10))`	`2`

Operadores para caracteres

El texto en R se manipula con funciones específicas.

Operación	Función	Ejemplo de Uso	Resultado
Concatenar	`paste()`	`paste("Curso", "de", "R")`	`"Curso de R"`
Concatenar	`paste0()`	`paste0("Curso", "de", "R")`	`"CursodeR"`
Contar car.	`nchar()`	`nchar(c("Hola", "Mundo"))`	`c(4, 5)`
Mayúsculas	`toupper()`	`toupper("hola")`	`"HOLA"`
Minúsculas	`tolower()`	`tolower("MUNDO")`	`"mundo"`
Subcadena	`substr()`	`substr("Viernes", 1, 4)`	`"Vier"`
Reemplazar	`gsub()`	`gsub("a", "o", "casa")`	`"coso"`
Dividir cad.	`strsplit()`	`strsplit("a-b", "-")`	`list(c("a", "b"))`
Eliminar esp.	`trimws()`	`trimws(" texto ")`	`"texto"`

Aritmética vectorizada: Operaciones elementales sin bucles

Esta es una de las características más potentes de R y lo diferencia de lenguajes como C o Python clásico. R está diseñado para operar sobre vectores completos de una sola vez (similar al concepto SIMD - Single Instruction, Multiple Data).

¿Por qué es importante?

1. Claridad: El código se lee como una operación matemática («precio por cantidad»), no como una instrucción informática («toma el elemento i, multiplícalo, guárdalo, repite»).
2. Eficiencia: Las operaciones vectorizadas están optimizadas en C, siendo mucho más rápidas que un bucle explícito escrito en R.

```r
# R
precios <- c(100, 200, 300)
precios_con_iva <- precios * 1.21
```

```julia
# Julia
precios = [100, 200, 300]
precios_con_iva = precios .* 1.21
```

```python
# Python
precios = [100, 200, 300]
precios_con_iva = [p * 1.21 for p in precios]
```

En R, también podés operar dos vectores entre sí. La operación se hace posición a posición (el 1ro con el 1ro, el 2do con el 2do, etc.).

```
ingreso  ← c(100, 200, 300)
costo    ← c(75, 125, 280)
utilidad ← ingreso - costo
```

```
[1] 25 75 20
```

Matrices

Las matrices son extensiones de los vectores a dos dimensiones (filas y columnas). Al igual que los vectores, todos sus elementos deben ser del mismo tipo (todos números, todos caracteres, etc.).

Creación de matrices

Usamos la función `matrix()`, especificando los datos y las dimensiones.

```r
# Crear una matriz de 3 filas y 2 columnas con números del 1 al 6
mi_matriz <- matrix(data = 1:6, nrow = 3, ncol = 2)
mi_matriz
```

```
     [,1] [,2]
[1,]    1    4
[2,]    2    5
[3,]    3    6
```

```r
# Por defecto llena por columnas. Para llenar por filas:
matriz_filas <- matrix(1:6, nrow = 3, byrow = TRUE)
```

Propiedades y acceso

Podemos consultar sus dimensiones y acceder a elementos específicos usando corchetes `[fila, columna]`.

```
dim(mi_matriz)      # Devuelve c(3, 2) -> cantidad de filas y de columnas
nrow(mi_matriz)     # Número de filas
ncol(mi_matriz)     # Número de columnas

# Indexación (Slicing)
mi_matriz[1, 2]     # Elemento en fila 1, columna 2
mi_matriz[2,  ]     # Toda la fila 2
mi_matriz[ , 1]     # Toda la columna 1

# O varias selecciones a la vez
mi_matriz[ , c(1, 5)]     # Error, porque la columna 5 no existe
```

Operaciones básicas

Al igual que con los vectores, las operaciones matemáticas son elemento a elemento. También existen funciones específicas para matrices.

```
mi_matriz * 2        # Multiplica cada elemento por 2
t(mi_matriz)         # Transpuesta (cambia filas por columnas)
rowSums(mi_matriz)   # Suma de cada fila
colMeans(mi_matriz)  # Promedio de cada columna
```

Acceso a elementos de un vector (indexación)

Una vez que tenemos datos en un vector, a menudo necesitamos seleccionar partes específicas de él. Para esto usamos los corchetes `[]`.

Indexación por posición (enteros positivos)

Indicamos la posición del elemento que queremos recuperar.

```r
letras <- c("A", "B", "C", "D", "E")

letras[1]       # El primer elemento ("A")
letras[3]       # El tercer elemento ("C")

# Podemos pedir varios a la vez pasando un vector de posiciones
letras[c(1, 5)] # El primero y el quinto ("A", "E")
```

Indexación por exclusión (enteros negativos)

Si usamos números negativos, R devuelve **todo menos** lo que indicamos.

```
letras ← c("A", "B", "C", "D", "E")

letras[-1]       # Todo menos el primero
letras[-c(4, 5)] # Todo menos el cuarto y el quinto
```

Indexación lógica (filtrado)

Esta es quizás la forma más potente. Usamos un vector de TRUE y FALSE (usualmente resultado de una condición) para quedarnos solo con los elementos que cumplen la condición (TRUE).

```
valores <- c(10, 5, 20, 2)

# Paso 1: Crear la condición (vector lógico)
condicion <- valores > 5
condicion
```

```
[1] TRUE FALSE TRUE FALSE
```

```
# Paso 2: Usar la condición para filtrar
valores[condicion]
```

```
[1] 10 20
```

```
# Todo en una sola línea (lo más común):
valores[valores > 5]
```

```
[1] 10 20
```

Incluso se puede hacer filtrados más complejos

```r
nombres ← c("Camila", "Pedro", "Lucía", "Juan", "Esteban")
notas ← c(10, 5, 3, 7, 8)

aprobo ← notas ≥ 6 #Vector lógico de aprobados/as

nombres[aprobo] #Filtrado por aprobados/as
```

```
[1] "Camila"  "Juan"    "Esteban"
```

Poder entender el funcionamiento de este mecanismo **es fundamental**!

Ejercicios

Intenta resolverlos paso a paso en un único script.

1. Crea un objeto con tu año de nacimiento y otro con el año actual (como número). Calcula tu edad aproximada restando ambos objetos y guárdala en una nueva variable.
2. Crea un vector con las temperaturas máximas de la semana: 24, 28, 31, 25, 22. Convierte todo el vector a grados Fahrenheit ($F = C \times 1.8 + 32$) en una sola línea.
3. Genera una secuencia de números que vaya del 100 al 10 bajando de 5 en 5. Guarda el resultado y calcula la suma de todos esos números.
4. Tienes un vector de precios: `c(100, 500, 200, 1500)`. Aplica un descuento del 10 % a todos, pero luego súmales un costo fijo de envío de 50 pesos a cada uno.
5. Crea un vector con los nombres de 4 ciudades y otro con sus respectivas poblaciones (en millones). Usa `paste()` para crear frases tipo «La ciudad X tiene Y millones de habitantes».
6. Dado el vector `x <- c(4, 9, 16, 25)`, calcula la raíz cuadrada de todos los elementos y luego calcula el promedio de esos resultados.
7. Crea un vector lógico que indique qué números del 1 al 20 son divisibles por 3. (Pista: usa el operador módulo `%%` y compara con 0).
8. Tienes dos vectores: `a <- c(TRUE, FALSE, TRUE)` y `b <- c(FALSE, TRUE, TRUE)`. Predice el resultado de `a & b` y `a | b` antes de ejecutarlo.

Ejercicios

9. Crea una matriz de 4 filas y 3 columnas con los números del 1 al 12. Calcula la suma de cada columna y guarda el resultado en un vector.
10. De la matriz anterior, extrae solamente la tercera fila y multiplícala por 10.
11. Crea un vector con 5 palabras en minúscula. Conviértelas todas a mayúsculas y luego cuenta cuántos caracteres tiene cada una.
12. Simula las notas de un examen para 6 alumnos con un vector numérico. Crea un vector lógico que sea `TRUE` para quienes aprobaron (nota = 6) y `FALSE` para los que no.
13. Usa el vector lógico del ejercicio anterior para contar cuántos alumnos aprobaron (Pista: en R, `TRUE` vale 1 y `FALSE` vale 0, así que puedes sumar el vector lógico).
14. Crea dos vectores de longitud 3: `base` y `altura`. Calcula el área de 3 triángulos simultáneamente.
15. Tienes un vector de edades: `c(15, 22, 35, 12, 45, 17)`. Filtra y muestra solo las edades que son mayores de edad y menores de 40.
16. Crea una matriz de 2x2 con valores aleatorios (puedes inventarlos). Calcula su transpuesta y verifíca si la suma de la matriz original más la transpuesta da una matriz simétrica.
17. Intenta sumar un vector de longitud 2 a un vector de longitud 5. Observa el resultado y la advertencia de R. ¿Cómo se aplicó el reciclaje?

Ejercicios

18. Crea un vector de textos con fechas en formato «DD-MM-AAAA»(ej: 01-01-2023). Usa `substr()` para extraer solo los años de cada fecha.
19. Genera una secuencia del 1 al 50. Crea un vector lógico que identifique los números que NO son divisibles por 5 (usando negación `!`).
20. Crea una matriz de 3x3 donde todos los elementos sean 0. Luego, modifica manualmente el elemento central (fila 2, columna 2) para que valga 1.
21. Tenés un vector de nombres `c("Juan", "Ana", "Pedro")` y un vector de apellidos `c("Pérez", "Gómez")`. Úsalos en `paste()`. ¿Qué sucede debido a la regla de reciclaje?
22. Calcula el logaritmo natural de los números 10, 100 y 1000. Luego redondea los resultados a 2 decimales.
23. Crea una matriz de 5 filas y 2 columnas. La primera columna debe tener los números del 1 al 5, y la segunda columna sus cuadrados. Asigna nombres a las columnas (Numero , Cuadrado) usando `colnames()`.